

Kazakh-British Technical University

Data collection and preparation

Laboratory work №2

NumPy & Pandas Foundations: NumPy arrays, Pandas indexing, filtering, and initial EDA

Full Name:

Baibolov Sultanmakmut

ID: 23B031235

08.09.2026

Almaty

Table of contents

Task 1. NumPy	3
Task 2. DataFrame	5
Task 3. Indexing.....	7
Task 4. Filtering	8
Task 5. EDA.....	9

Task 1. NumPy

Create a two-dimensional NumPy array and use it to demonstrate the main array operations.

Inspect its structure using shape, ndim, size, and dtype, then practice indexing and slicing by selecting rows, columns, and smaller parts of the array.

Calculate basic statistics for the whole array and along different axes. Perform several vectorized operations without using Python loops. Show the difference between axis=0 and axis=1.

I used the random module and created a new generator to initialize a 5x5 two-dimensional array. Although the numbers are random (from -10 to 10), the array contains the same set of numbers each time it's run.

Setting seed for random number generator

```
[2]: rng = npr.default_rng(seed=67)
```

Task 1. NumPy

Random 2-d array of integers

```
[3]: arr = rng.integers(low=-10, high=10, size=(5, 5))
```

```
[4]: arr
```

```
[4]: array([[ 5, -2,  1, -8, -2],
           [ 0,  2, -5,  4,  1],
           [ 3, -2,  7, -4, -8],
           [-3, -2, -9, -7,  8],
           [ 0, -6, -3, -10,  5]])
```

Shape, ndim, size, and dtype

```
[5]: arr.shape
```

```
[5]: (5, 5)
```

```
[6]: arr.ndim
```

```
[6]: 2
```

```
[7]: arr.size
```

```
[7]: 25
```

```
[8]: arr.dtype
```

```
[8]: dtype('int64')
```

Next, I used attributes to show the size, the number of dimensions, the count of elements, and the data type.

Indexing and slicing

Basic indexing and slicing

```
[9]: arr
[9]: array([[ 5, -2,  1, -8, -2],
          [ 0,  2, -5,  4,  1],
          [ 3, -2,  7, -4, -8],
          [-3, -2, -9, -7,  8],
          [ 0, -6, -3, -10,  5]])
[10]: arr[0]
[10]: array([ 5, -2,  1, -8, -2])
[11]: arr[0, 0]
[11]: np.int64(5)
[12]: arr[:, ]
[12]: array([[ 5, -2,  1, -8, -2],
          [ 0,  2, -5,  4,  1],
          [ 3, -2,  7, -4, -8],
          [-3, -2, -9, -7,  8],
          [ 0, -6, -3, -10,  5]])
[13]: arr[0:2]
[13]: array([[ 5, -2,  1, -8, -2],
          [ 0,  2, -5,  4,  1]])
[14]: arr[:2]
[14]: array([[ 5, -2,  1, -8, -2],
          [ 0,  2, -5,  4,  1]])
[15]: arr[:, 2:]
[15]: array([[ 1, -8, -2],
          [-5,  4,  1]])
[16]: arr[:, 0]
[16]: array([ 5,  0,  3, -3,  0])
```

Boolean indexing

Indexing using boolean arrays

```
[17]: arr
[17]: array([[ 5, -2,  1, -8, -2],
          [ 0,  2, -5,  4,  1],
          [ 3, -2,  7, -4, -8],
          [-3, -2, -9, -7,  8],
          [ 0, -6, -3, -10,  5]])
[18]: arr_copy = arr.copy()
[19]: arr_copy > 0
[19]: array([[ True, False,  True, False, False],
          [False,  True, False,  True,  True],
          [ True, False,  True, False, False],
          [False, False, False, False,  True],
          [False, False, False, False,  True]])
[20]: arr_copy[arr_copy > 0]
[20]: array([5, 1, 2, 4, 1, 3, 7, 8, 5])
[34]: arr.mean()
[34]: np.float64(-1.4)
[35]: arr.mean(axis=0)
[35]: array([ 1. , -2. , -1.8, -5. ,  0.8])
[36]: arr.mean(axis=1)
[36]: array([-1.2,  0.4, -0.8, -2.6, -2.8])
[37]: arr.sum()
[37]: np.int64(-35)
[38]: arr.sum(axis=0)
[38]: array([ 5, -10, -9, -25,  4])
[39]: arr.sum(axis=1)
[39]: array([-6,  2, -4, -13, -14])
```

Fancy indexing

Indexing using integer arrays

```
[25]: arr
[25]: array([[ 5, -2,  1, -8, -2],
          [ 0,  2, -5,  4,  1],
          [ 3, -2,  7, -4, -8],
          [-3, -2, -9, -7,  8],
          [ 0, -6, -3, -10,  5]])
[26]: arr[[0, 2, 4]]
[26]: array([[ 5, -2,  1, -8, -2],
          [ 3, -2,  7, -4, -8],
          [ 0, -6, -3, -10,  5]])
[27]: arr[-1, -3, -5]
[27]: array([[ 0, -6, -3, -10,  5],
          [ 3, -2,  7, -4, -8],
          [ 5, -2,  1, -8, -2]])
[28]: arr[[0, 2, 4], [0, 2, 4]]
[28]: array([5, 7, 5])
```

Vectorized operations

```
[47]: np.square(arr)
[47]: array([[ 25,  4,  1,  64,  4],
          [  0,  4, 25,  16,  1],
          [  9,  4, 49,  16,  64],
          [  9,  4, 81,  49,  64],
          [  0, 36,  9, 100, 25]])
[48]: np.exp(arr)
[48]: array([[1.48413159e+02, 1.35335283e-01, 2.71828183e+00, 3.35462628e-04,
          1.35335283e-01],
          [1.00000000e+00, 7.38905610e+00, 6.73794700e-03, 5.45981500e+01,
          2.71828183e+00],
          [2.00855369e+01, 1.35335283e-01, 1.09663316e+03, 1.83156389e-02,
          3.35462628e-04],
          [4.97870684e-02, 1.35335283e-01, 1.23409804e-04, 9.11881966e-04,
          2.98095799e+03],
          [1.00000000e+00, 2.47875218e-03, 4.97870684e-02, 4.53999298e-05,
          1.48413159e+02]])
```

I used three types of indexing: basic, Boolean, and a special type with an array specified as the

index. Basic indexing is similar to indexing with a list from the basic library, but instead of two pairs of square brackets, you need to specify the indices separated by commas. With Boolean indexing, we use an array with true/false values of the same length as the original array. All elements corresponding to true will be returned. There is also the option to use arrays of indices to return data in a specific order.

Task 2. DataFrame

Choose a structured dataset with at least 100 rows and 5 columns that contain both numerical and categorical data. Load it into a Pandas DataFrame and inspect its structure. Determine the dataset size, column names, data types, variables, and whether missing values are present. Use appropriate Pandas methods to obtain basic descriptive information. State what one row of the dataset represents.

I chose the Titanic dataset in CSV format.

```
[52]: df = pd.read_csv("../archive/Titanic-Dataset.csv")
```

```
[53]: df.head()
```

```
[53]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Basic information

```
[54]: df.size
```

```
[54]: 10692
```

```
[55]: df.shape
```

```
[55]: (891, 12)
```

```
[56]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    str
4   Sex          891 non-null    str
5   Age          714 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    str
9   Fare         891 non-null    float64
10  Cabin        204 non-null    str
11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

Using the info method, you can immediately find out all the necessary information to start working with the data frame.

Basic descriptive information

```
[57]: df.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Also, using the describe method, which will provide all the necessary statistics for columns with a numeric data type.

```
[58]: df.iloc[0]
```

```
[58]: PassengerId      1
      Survived        0
      Pclass         3
      Name      Braund, Mr. Owen Harris
      Sex            male
      Age           22.0
      SibSp          1
      Parch          0
      Ticket      A/5 21171
      Fare           7.25
      Cabin         NaN
      Embarked      S
      Name: 0, dtype: object
```

1. **PassengerId** - Passenger ID
2. **Survived** - Weather Survived or not: 0 = No, 1 = Yes
3. **Pclass** - Ticket class: 1 = 1st, 2 = 2nd, 3 = 3rd
4. **Name** - Name of the Passenger
5. **Sex** - Gender
6. **Age** - Age in Years
7. **SibSp** - No. of siblings / spouses aboard the Titanic
8. **Parch** - No. of parents / children aboard the Titanic
9. **Ticket** - Ticket number
10. **Fare** - Passenger fare
11. **Cabin** - Cabin number
12. **Embarked** - Port of Embarkation: C = Cherbourg, Q = Queenstown, S = Southampton

Example of 1 line from the dataset.

Task 3. Indexing

Practice selecting data from the DataFrame using different Pandas indexing methods. Use `loc` and `iloc` to select rows, columns, ranges of data, and individual values. Create several meaningful selection examples based on your dataset. Show the difference between `loc` and `iloc`.

```
[62]: df['PassengerId']
```

```
[62]: 0      1
      1      2
      2      3
      3      4
      4      5
      ...
      886    887
      887    888
      888    889
      889    890
      890    891
      Name: PassengerId, Length: 891, dtype: int64
```

```
[63]: df[['Name', 'Sex', 'Survived']].head()
```

```
[63]:
```

	Name	Sex	Survived
0	Braund, Mr. Owen Harris	male	0
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	1
2	Heikkinen, Miss. Laina	female	1
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	1
4	Allen, Mr. William Henry	male	0

```
[64]: df.iloc[0:4]
```

```
[64]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S

```
[65]: df.iloc[0:10:2]
```

```
[65]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
8	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)	female	27.0	0	2	347742	11.1333	NaN	S

```
[67]: df.iloc[0, 0]
[67]: np.int64(1)
[68]: df_head_reindexed = df.head().reindex([4, 3, 2, 1, 0])
[69]: df_head_reindexed
[69]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S

```

[70]: df_head_reindexed.iloc[0, 3]
[70]: 'Allen, Mr. William Henry'
[71]: df_head_reindexed.loc[0, 'Name']
[71]: 'Braund, Mr. Owen Harris'

```

To demonstrate the difference between `iloc` and `loc`, I created a copy of the first 5 rows and reindexed them in reverse order. The difference between the two approaches is that `iloc` returns rows based on their position in the Series, starting from 0. `Loc`, on the other hand, returns rows based on labels, the Index object, and the Series (columns).

Task 4. Filtering

Formulate four meaningful questions about your dataset and answer them using Pandas filtering. Use numerical and categorical conditions, a combination of multiple conditions, and at least one condition based on a calculated value such as the mean or median. For each case, show the question, filtering code, and resulting data.

I selected 4 questions that fit the requirements of the task and answered them.

Questions

1. How many passengers were adults?
2. How many passengers were embarked in Queenstown?
3. How many woman passengers survived?
4. Which passengers had fare higher than the median?

Question 1: How many passengers were adults?

```
[72]: (df[df['Age'] >= 18].count()).iloc[0]
[72]: np.int64(601)
```

Question 2: How many passengers were embarked in Queenstown?

```
[73]: df[df['Embarked'] == 'Q'].count().iloc[0]
[73]: np.int64(77)
```

Question 3: How many woman passengers survived?

```
[74]: df[(df['Sex'] == 'female') & (df['Survived'] == 1)].count().iloc[0]
[74]: np.int64(233)
```

Question 4: Which passengers had fare higher than the median?

```
[75]: print(f"Median: {df['Fare'].median()}")
      df[df['Fare'] > df['Fare'].median()]
```

Median: 14.4542

```
[75]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
6	7	0	1	McCarthy, Mr. Timothy J	male	54.0	0	0	17463	51.8625	E46	S
7	8	0	3	Palsson, Master. Gosta Leonard	male	2.0	3	1	349909	21.0750	NaN	S
9	10	1	2	Nasser, Mrs. Nicholas (Adele Achem)	female	14.0	1	0	237736	30.0708	NaN	C
...	...	...	...	...	...	...	...	...	...	...	...	...
880	881	1	2	Shelley, Mrs. William (Imanita Parrish Hall)	female	25.0	0	1	230433	26.0000	NaN	S
885	886	0	3	Rice, Mrs. William (Margaret Norton)	female	39.0	0	5	382652	29.1250	NaN	Q
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.0000	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.4500	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.0000	C148	C

444 rows x 12 columns

Task 5. EDA

Perform an initial exploration analysis of the dataset. Check data quality, examine the distributions of important numerical and categorical variables, and investigate at least one possible relationship between numerical features. Use missing-value checks, duplicate detection, value_counts(), descriptive statistics, and correlation where appropriate. Write five specific findings based on the results.

First, I cleaned the dataframe by removing missing values in the age columns and replacing them with the median. Since more than half of the values in the Cabin column are missing, I deleted it. I also deleted the remaining 2 rows with an unspecified boarding location. After that, I checked that there were no duplicates in the dataset and proceeded to evaluate the categorical and numerical data.

Missing values and duplicates

```
[76]: df_clear = df
```

```
[77]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype  
---  --
 0   PassengerId    891 non-null    int64  
 1   Survived       891 non-null    int64  
 2   Pclass        891 non-null    int64  
 3   Name          891 non-null    str     
 4   Sex           891 non-null    str     
 5   Age           714 non-null    float64 
 6   SibSp         891 non-null    int64  
 7   Parch         891 non-null    int64  
 8   Ticket        891 non-null    str     
 9   Fare          891 non-null    float64 
10  Cabin         204 non-null    str     
11  Embarked      889 non-null    str     
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
```

```
[78]: df_clear['Age'] = df['Age'].fillna(df['Age'].median())
```

```
[79]: df_clear['Age'].isna().sum()
```

```
[79]: np.int64(0)
```

```
[80]: df_clear = df_clear.drop(columns=['Cabin'])
```

```
[81]: df_clear.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    str
4   Sex          891 non-null    str
5   Age         891 non-null    float64
6   SibSp        891 non-null    int64
7   Parch        891 non-null    int64
8   Ticket       891 non-null    str
9   Fare         891 non-null    float64
10  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(4)
memory usage: 76.7 KB
```

```
[82]: df_clear = df_clear[df_clear['Embarked'].notna()]
```

```
[83]: df_clear.info()
```

```
<class 'pandas.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  889 non-null    int64
1   Survived     889 non-null    int64
2   Pclass       889 non-null    int64
3   Name         889 non-null    str
4   Sex          889 non-null    str
5   Age         889 non-null    float64
6   SibSp        889 non-null    int64
7   Parch        889 non-null    int64
8   Ticket       889 non-null    str
9   Fare         889 non-null    float64
10  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(4)
memory usage: 83.3 KB
```

```
[84]: df_clear.duplicated().sum()
```

```
[84]: np.int64(0)
```

▼ Categorical values statistics

```
[86]: categories = ['Survived', 'Pclass', 'Sex', 'Embarked']

for category in categories:
    df_clear[category] = df_clear[category].astype('category')
```

```
[87]: df_clear[categories].describe()
```

```
[87]:
```

	Survived	Pclass	Sex	Embarked
count	889	889	889	889
unique	2	3	2	3
top	0	3	male	S
freq	549	491	577	644

Numerical values statistics

```
[85]: df_clear[['Age', 'SibSp', 'Parch', 'Fare']].describe()
```

```
[85]:
```

	Age	SibSp	Parch	Fare
count	889.000000	889.000000	889.000000	889.000000
mean	29.315152	0.524184	0.382452	32.096681
std	12.984932	1.103705	0.806761	49.697504
min	0.420000	0.000000	0.000000	0.000000
25%	22.000000	0.000000	0.000000	7.895800
50%	28.000000	0.000000	0.000000	14.454200
75%	35.000000	1.000000	0.000000	31.000000
max	80.000000	8.000000	6.000000	512.329200

Relationships

```
[88]: df_clear[['Age', 'Fare', 'Pclass', 'Survived']].corr()
```

```
[88]:
```

	Age	Fare	Pclass	Survived
Age	1.000000	0.093707	-0.336512	-0.069822
Fare	0.093707	1.000000	-0.548193	0.255290
Pclass	-0.336512	-0.548193	1.000000	-0.335549
Survived	-0.069822	0.255290	-0.335549	1.000000

```
[89]: df_clear.groupby('Sex')['Survived'].value_counts()
```

```
[89]:
```

Sex	Survived	
female	1	231
	0	81
male	0	468
	1	109

Name: count, dtype: int64

I used a correlation matrix to determine whether there is a relationship between the variables, and I also compared whether survival is dependent on gender.

Specific findings

1. The Cabin column contained the greatest number of missing values: 687.
2. There were none duplicate entries.
3. The average fare was 32, with the median of 14.
4. Female passengers had survival rate of 74%, while male passengers only 19%.
5. The only pretty significant correlation was found between Fare and Pclass (negative

correlation with value -0.55).